perplexity

la pubblicazione della porta 8000, come definito in Python/templates/docker_run_extra_args e' effettivamente un refuso. Quando un progetto richiede la pubblicazione di una porta, a prescindere se sia avviato in modalita' legacy (singolo lang con run.sh) oppure tramite compose, abbiamo identificato una modalita' coerente per dichiarare l'intenzione ?

Non ancora in modo completo: avevamo identificato il requisito di una dichiarazione coerente, ma non avevamo fissato una fonte comune utilizzabile da entrambi i backend. Gli extra args coprono il percorso legacy, mentre `compose.user.yaml` copre Compose: mantenere la stessa intenzione in due file distinti introdurrebbe duplicazione.

Recepisco quindi la correzione: la porta 8000 non è un requisito da preservare. La regola diventa **nessuna pubblicazione implicita**; ogni progetto dichiara esplicitamente le porte che vuole rendere accessibili dall'host.

Dichiarazione comune del progetto

Proporrei un file di configurazione indipendente dal backend:

```
<PROJECT_DIR>/.docker-dev/project.yaml
```

Per esempio:

```
schema_version: 1

runtime:
  ports:
    - target: 8000
      published: "8080"
      host_ip: "127.0.0.1"
      protocol: tcp
```

Questa dichiarazione significa:

```
127.0.0.1:8080 sull'host → porta TCP 8000 del container di sviluppo
```

I campi riprendono la terminologia Compose: `target` identifica la porta nel container, `published` quella sull'host, `host_ip` l'indirizzo di binding e `protocol` il protocollo.[1]

L'assenza di `runtime.ports`, oppure una lista vuota, significa che il progetto non richiede alcuna pubblicazione:

```
schema_version: 1

runtime:
  ports: []
```

Il nome del file e lo schema sono una nuova proposta da approvare, non una convenzione già implementata.

Traduzione nei due backend

Il resolver legge e valida la configurazione una sola volta, producendo un modello normalizzato. I due backend traducono poi la stessa intenzione.

Modalità legacy

La dichiarazione precedente genera argomenti nell'array passato a `docker run`:

```
publish_args=(
  --publish "127.0.0.1:8080:8000/tcp"
)

docker run \
  "${publish_args[@]}" \
  ... \
  "$IMAGE_NAME" \
  "${CONTAINER_CMD[@]}"
```

Docker pubblica le porte con `--publish`/`-p`, usando il formato indirizzo host, porta host e porta container.[2] [3]

Modalità Compose

La stessa dichiarazione genera:

```
services:
  dev:
    ports:
      - target: 8000
        published: "8080"
        host_ip: "127.0.0.1"
        protocol: tcp
```

Questa è la sintassi estesa prevista da Compose per la pubblicazione delle porte.[1]

Dato che abbiamo scelto un container `dev` one-off, il comando di esecuzione deve anche usare:

```
docker compose "${compose_args[@]}" \
  run --rm --service-ports dev "${CONTAINER_CMD[@]}"
```

`--service-ports` fa sì che `compose run` pubblichi le porte dichiarate nel servizio; senza tale opzione la sola sezione `ports` non basta per il container one-off. [4]

Dove collocare questa responsabilità

Questa configurazione non dovrebbe appartenere al resolver dei soli servizi: riguarda il container di sviluppo anche quando non viene richiesto alcun servizio aggiuntivo.

La separazione proposta è:

```
configurazione progetto
       ↓
resolver comune del runtime
       ├─ backend legacy → array argomenti docker run
       └─ backend Compose → configurazione services.dev

catalogo services/
       ↓
resolver servizi → configurazione dei servizi di supporto
```

`40.run.sh` usa il runtime già risolto. `30.services.sh` continua invece a occuparsi dei servizi opzionali e della preparazione Compose, senza diventare l'unico punto capace di interpretare le porte del progetto.

Interazione con gli extra args esistenti

Per evitare due fonti concorrenti, proporrei questa politica:

Fonte	Ruolo
`project.yaml`	Dichiarazione canonica delle porte del progetto, comune ai backend
`docker-run-extra-args`	Compatibilità legacy per altre opzioni
`compose.user.yaml`	Personalizzazioni specifiche Compose, non sede ordinaria delle porte di `dev`

La transizione va però definita esplicitamente:

- rimuovere `-p 8000:8000` dal template Python centrale;
- non modificare automaticamente i file già presenti nei progetti: alcune pubblicazioni potrebbero essere intenzionali;
- se un progetto dichiara porte nel nuovo file e anche negli extra args, segnalare il conflitto;
- durante la migrazione, le porte presenti solo negli extra args possono continuare a funzionare in legacy, ma non vanno considerate automaticamente portabili a Compose.

Un traduttore generale degli extra args non è necessario: è più semplice introdurre una dichiarazione strutturata per le sole funzionalità che devono essere comuni.

Pubblicazione verso l'host e comunicazione tra servizi

Va mantenuta distinta l'intenzione "accedere dal computer host" da "comunicare con un altro container".

Per esempio, il container di sviluppo può collegarsi a PostgreSQL tramite:

```
db:5432
```

senza pubblicare la porta del database sull'host. La comunicazione tra servizi Compose usa il nome del servizio e la porta interna; la porta pubblicata serve per l'accesso dall'esterno della rete dello stack. [5]

Di conseguenza:

- il catalogo PostgreSQL può conoscere la porta interna 5432;
- tale conoscenza non deve implicare una pubblicazione;
- la pubblicazione di un servizio di supporto deve essere richiesta dalla configurazione del progetto.

Scelte da fissare nella CR

La soluzione comune può essere consolidata con questi punti:

1. File canonico: proposta `.docker-dev/project.yaml`.
2. Nessuna porta pubblicata per default.
3. Backend legacy e Compose derivano la pubblicazione dallo stesso modello.
4. Binding: proposta `127.0.0.1` come default quando la pubblicazione è richiesta; accesso su tutte le interfacce solo esplicito. Docker, senza indirizzo host, pubblica normalmente su tutte le interfacce. [2]
5. Prima versione: porte singole TCP/UDP con porta host esplicita; porte dinamiche e intervalli possono essere aggiunti successivamente.
6. Conflitti fra configurazione comune e personalizzazioni backend: errore esplicito, non precedenza silenziosa.

Questo sostituisce nella CR il requisito "preservare la porta 8000" con quello più corretto: dichiarare una sola volta l'intenzione di pubblicazione e applicarla coerentemente al backend selezionato.

⁂

1. https://docs.docker.com/reference/compose-file/services.md
2. https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/

3. https://docs.docker.com/engine/network/port-publishing/
4. https://docs.docker.com/reference/cli/docker/compose/run/
5. https://docs.docker.com/compose/how-tos/networking/
6. https://docs.docker.com/reference/glossary/
7. https://docs.docker.com/ai/sandboxes/workflows.md
8. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-docker-compose/
9. https://docs.docker.com/reference/compose-file/services/
10. https://docs.docker.com/reference/cli/docker/compose/
11. https://docs.docker.com/compose/intro/compose-application-model/
12. https://docs.docker.com/reference/cli/docker/compose/port/
13. https://docs.docker.com/reference/cli/docker/compose/ps/
14. https://docs.docker.com/engine/swarm/networking/
15. https://docs.docker.com/reference/cli/docker/container/port/